Plant Disease Classification (Mroot)

The goal of this project was to build a model that predicts plant diseases from images while ignoring the plant type. For example, both “tomato late blight” and “potato late blight” should be predicted as “late_blight”.
To match this idea, the dataset was organized into disease-only classes. In the end we had 39 classes.
For the model I used EfficientNet-B0. The setup was image size 288x288, weighted sampling because the dataset is imbalanced, MixUp augmentation, and AdamW optimizer. This combination worked pretty stable overall.
I also tried a bunch of other things during the process. For example, I tested different architectures like ResNet34 and EfficientNet-B1. I also tried Vision Transformer, but it overfit really fast and didn’t behave well on this dataset, so I didn’t continue with it.

One thing I noticed was that the model often confused diseases that look very similar (especially leaf spot types and rust-like diseases). So I slightly reduced MixUp (from 0.2 to 0.1) so the model keeps more original details from the image. That helped a bit.

I also experimented with some preprocessing ideas. I tried segmentation-like approaches, where I focused more on the leaf area, and also background removal to get rid of unnecessary parts of the image. The idea was that the model should focus only on the important regions. But in practice, this didn’t really improve the results. In some cases it even made things worse, probably because the model was already able to learn useful features without manually removing background.

Another idea I tried was grouping classes and training multiple models (like splitting similar diseases), but this turned out to be more complicated and didn’t perform better than just using one good model, so I dropped it.

During experimments, I saw slightly higher scores (around 0.89 mAP) when using the original validation setup. Later, I created a stricter train/validation/test split, and with that the results became a bit lower (~0.88 on validation and ~0.86 on test), but I think these are more realistic and reliable.

Overall, the main takeaway for me was that getting the setup right and focusing on generalization mattered more than trying too many complicated ideas. In the end, a well-tuned EfficientNet-B0 worked better than more complex approaches.